

Langage Python pour les programmeurs Java et C#

rouanehm@gmail.com



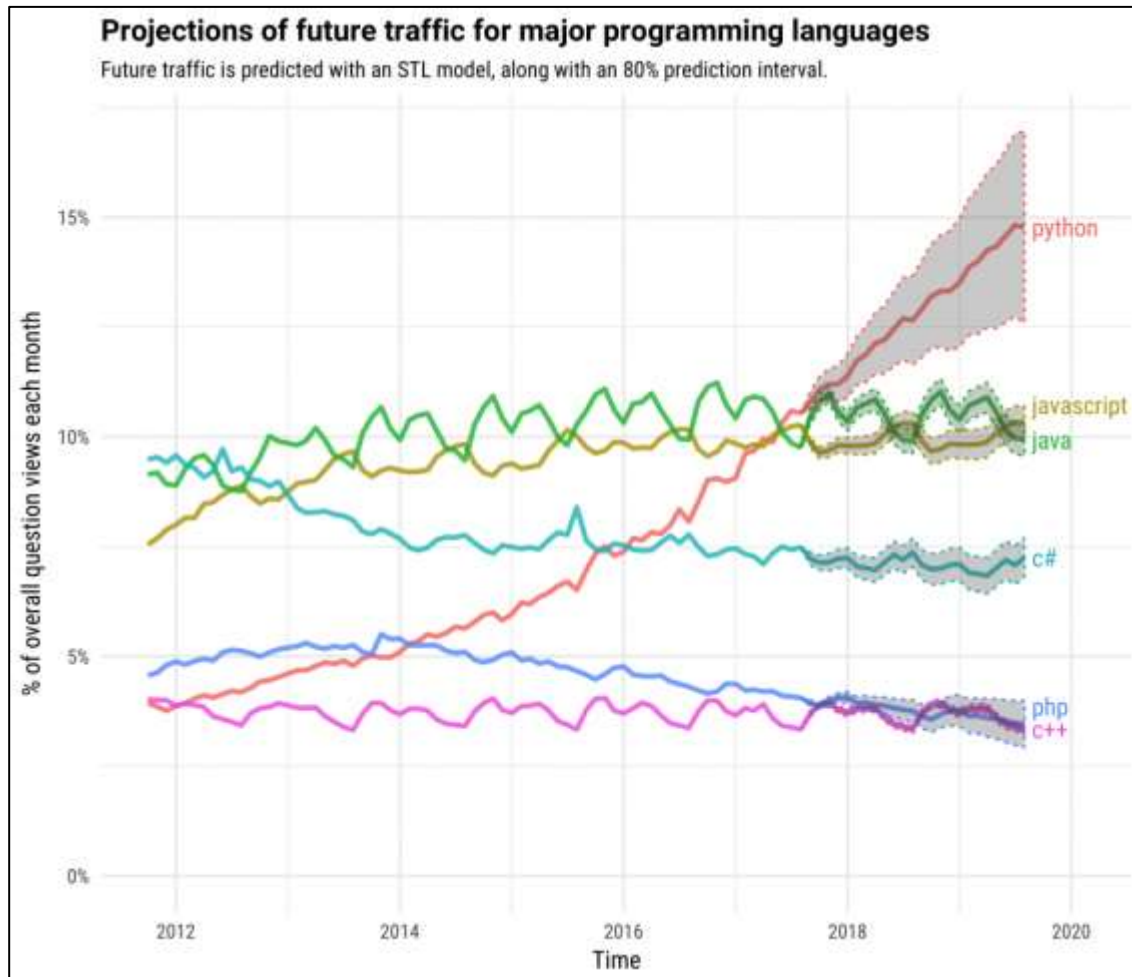
Python

- Un langage de programmation multiparadigme.
- Né en 1990
- Dernière version 3.8 (Mai 2020)
- Multiplateformes
- Architectures variées: Desktop, Web (Flask, Django) et Mobile (Flutter, Kivy, BeeWare)
- Python developers are in extensive demand in diverse industries such as machine learning, databases, cloud infrastructure, design, data analysis, webpage reliability/testing, web scraping, security, mobile development, etc.

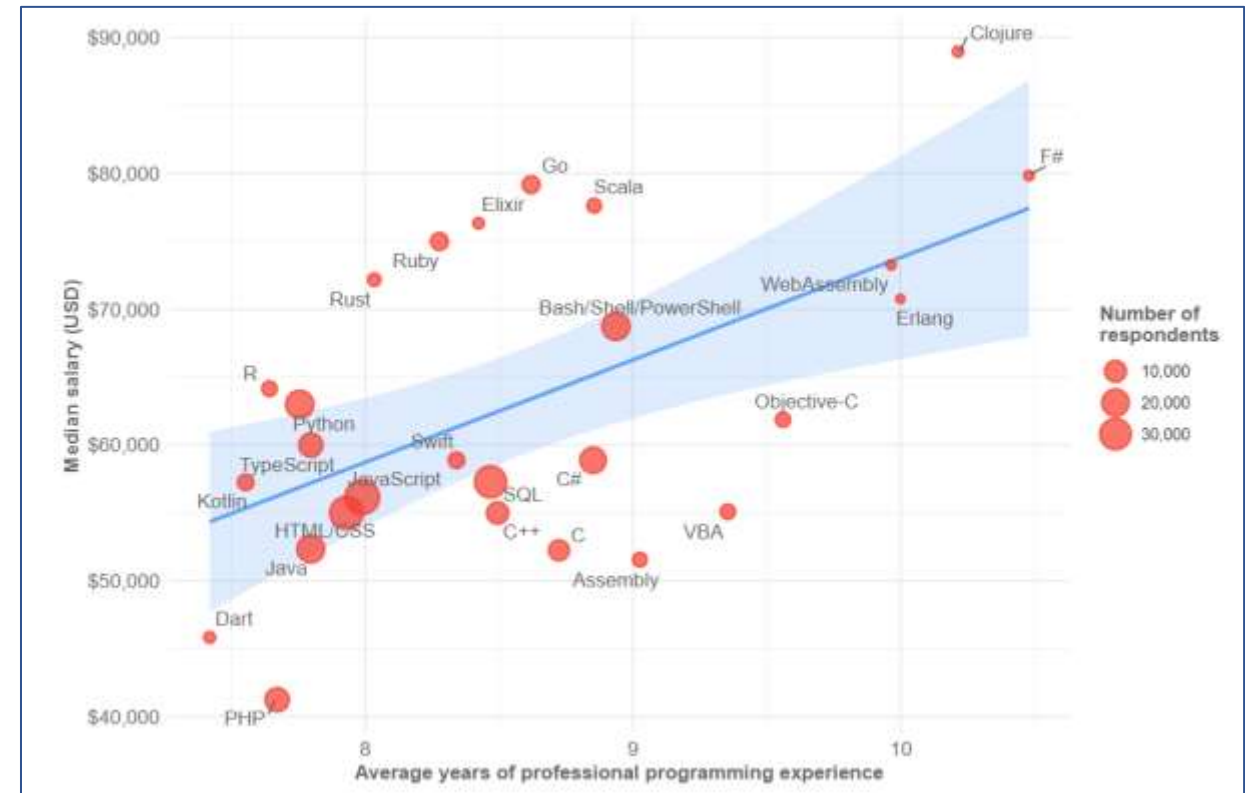
 python ™	
Paradigm	Multi-paradigm: functional, imperative, object-oriented, structured, reflective
Designed by	Guido van Rossum
Developer	Python Software Foundation
First appeared	1990; 30 years ago ^[1]
Stable release	3.8.5 / 20 July 2020; 35 days ago ^[2]
Preview release	3.9.0rc1 / 11 August 2020; 13 days ago ^[3]
Typing discipline	Duck, dynamic, gradual (since 3.5) ^[4]
OS	Linux, macOS, Windows Vista (and newer) and more
License	Python Software Foundation License
Filename extensions	.py, .pyi, .pyc, .pyd, .pyo (prior to 3.5), ^[5] .pyw, .pyz (since 3.5) ^[6]
Website	www.python.org 

Position de Python :

The fastest-growing major programming language (Stackoverflow)



<https://stackoverflow.blog/2017/09/06/incredible-growth-python/>



Salaire moyen développeur Python au Canada: 75K\$

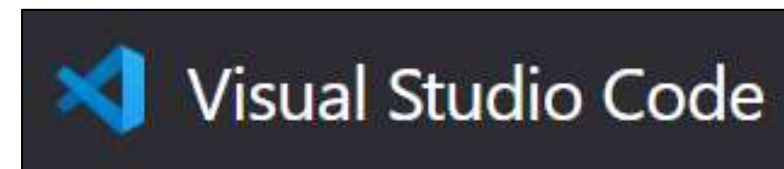
<https://neuvoo.ca/salaire/?job=Developpeur%20Python>

Quelques utilisateurs de Python



IDE Python

- PyCharm (Jetbrains, IntelliJ IDEA) – **IDE officiel du cours** –
 - <https://www.youtube.com/watch?v=56bPIGf4us0>
 - <https://www.youtube.com/watch?v=Jt0fNgJr400>
- Spyder
 - <https://www.spyder-ide.org/>
- Sublime Text 3
 - <https://www.sublimetext.com/3>
- Atom
 - <https://atom.io/packages/ide-python>
- VS Code
 - <https://code.visualstudio.com/docs/languages/python>
- Online Compilers
 - <https://www.programiz.com/python-programming/online-compiler/>

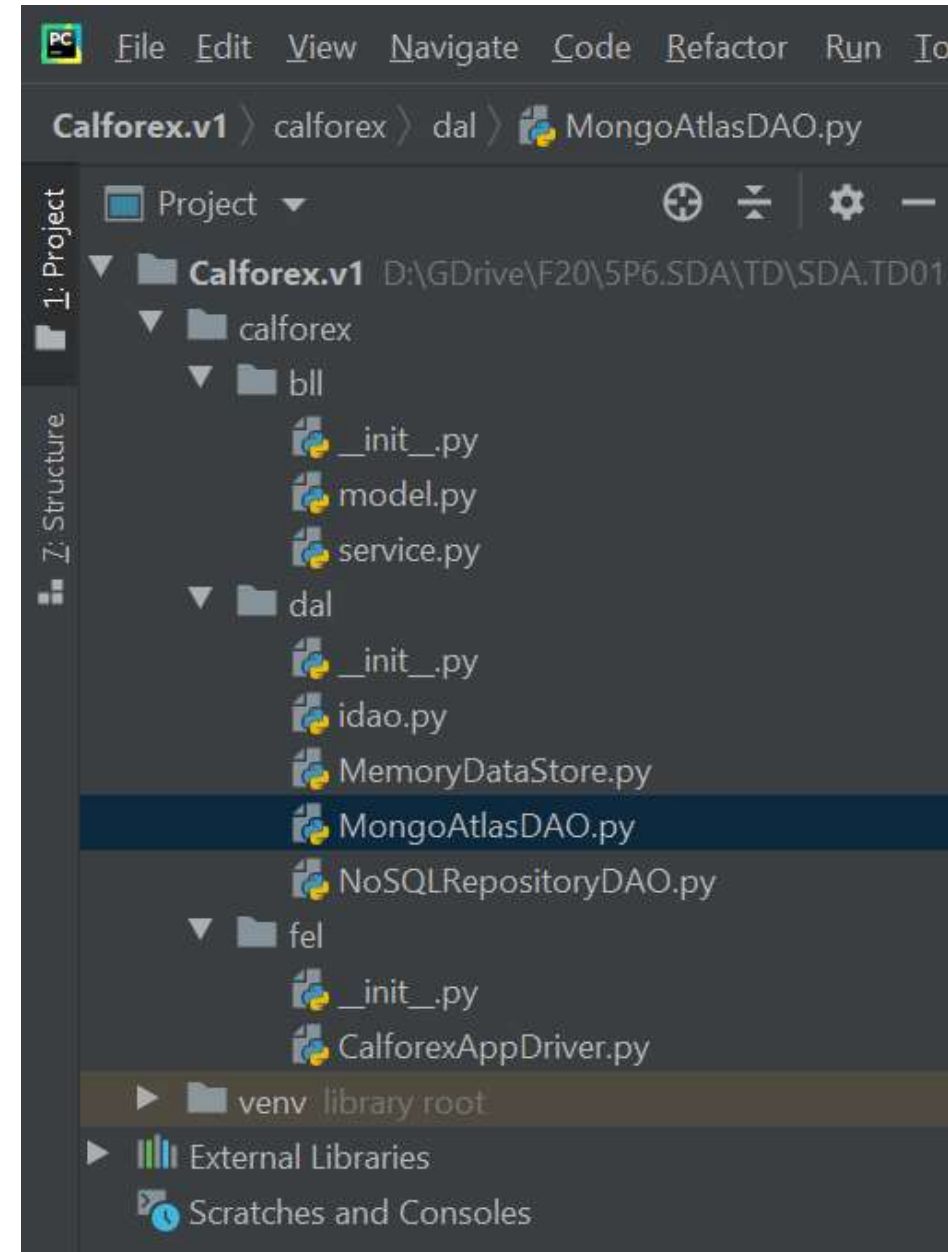


Structure d'un projet Python

Le module

- Module: un fichier de code Python
- Un module peut contenir plusieurs classes ou simplement des fonctions.
- Pour utiliser les définitions déjà faites, il faut utiliser le mot-clé `import`.

```
1 # --- Developer: Amine 2020-06-13 23:14
2 import pymongo as pymongo
3
4 from calforex.bll.model import Rate
5 from calforex.dal.idao import IRateDAO, ICurrencyDAO
6 from calforex.dal.NoSQLRepositoryDAO import NoSQLRepositoryDao
7
8
9 class MongoAtlasDAO(NoSQLRepositoryDao, IRateDAO, ICurrencyDAO):
```



Structure d'un projet Python: Importer un module

- Python lui-même est bâti sur des centaines de modules qui réalisent diverses tâches (math, re, os, time, urllib, etc.) qui sont préinstallés.
- Pour utiliser un module, il faut l'importer.
- On peut importer un élément spécifique d'un module ou tous les éléments d'un module.
- Il est possible de définir un alias pour un module afin de raccourcir son nom ou lui donner un nouveau nom significatif dans son propre code.
- Pour vider de la mémoire d'un module déjà chargé, on peut utiliser l'instruction del.

```
1 >>> import random
2 >>> random.randint(0,10)
3 2
4 >>> del random
```

```
1 >>> from random import randint
2 >>> randint(0,10)
3 7
```

```
1 >>> from random import *
2 >>> x = [1, 2, 3, 4]
3 >>> shuffle(x)
4 >>> x
5 [2, 3, 1, 4]
6 >>> shuffle(x)
7 >>> x
8 [4, 2, 1, 3]
9 >>> randint(0,50)
10 46
11 >>> uniform(0,2.5)
12 0.64943174760727951
```

```
1 import random
1 random.randint(0,2)

1 >>> import random as rand
2 >>> rand.randint(1, 10)
3 6
4 >>> rand.uniform(1, 3)
5 2.643472616544236
```

Structure d'un projet Python:

Ajouter un fichier `__init__.py`

- Un Package Python est un répertoire qui contient un ensemble de modules Python. Rappelez-vous, un module est tout simplement un fichier d'instructions Python.
- Ajouter un fichier `__init__.py` dans un répertoire Python permet d'importer tous les modules indiqués dans ce fichier ce qui réduit le nombre d'imports que l'on pourrait mettre au début d'un module.

```
import string_func.stringLength
import string_func.stringToLower
import string_func.stringToUpper

some_string = "Hello, Universe!"

print(string_func.stringLength.stringLength(some_string))
print(string_func.stringToLower.stringToLower(some_string))
print(string_func.stringToUpper.stringToUpper(some_string))
```



```
someFolder
|-- stringLength.py
|-- stringToUpper.py
`-- stringToLower.py
```

```
someFolder
|-- string_func
|   |-- __init__.py
|   |-- stringToUpper.py
|   |-- stringToLower.py
|   `-- strengthLength.py
`-- example1.py
```

```
# __init__.py
from .stringLength import stringLength
from .stringToLower import stringToLower
from .stringToUpper import stringToUpper
```

```
import string_func

some_string = "Hello, Universe!"

print(string_func.stringLength(some_string))
print(string_func.stringToLower(some_string))
print(string_func.stringToUpper(some_string))
```

La classe

- **From/import** pour inclure d'autres définitions de classes ou de code.
- Mot clé **self** est équivalent à Java **this**
- Héritage est indiqué avec le nom de la classe et peut être multiple.
- Fonction **__init__** est le constructeur. On peut surcharger le constructeur.
- Les variable précédés par **self** dans **__init__** sont des variable d'instance.
- Les variables hors **__init__** sont des variables de classe.

```
1      # --- Developer: Amine 2020-06-13 16:21
2      from calforex.bll.model import Rate
3      from calforex.dal.idao import IRateDAO, ICurrencyDAO
4
5
6      class MemoryDataStore(IRateDAO, ICurrencyDAO):
7
8          def __init__(self):
9              self.rates = []
10             self.populate()
11
12             def fetch_all_rates(self) -> []:
13                 """Overrides IExchangeDAO.fetch_all_rates()"""
14                 return [rate for rate in self.rates]
15
16             def fetch_rate(self, source: str, target: str) -> float:
17                 """Overrides IExchangeDAO.fetch_rate()"""
18                 for rate in self.rates:
19                     if rate.source == source and rate.target == target:
20                         return rate.value
21                 return 0.0
```

La classe:

Les modificateurs d'accès

- Comme tout langage de POO, Python possède les trois modes d'accès public, protected (__) et private (__). Les noms des membres privés et protégés commencent par _/_ mais ne se terminent pas avec ces symboles (à l'instar des dunder methods)

```
class Car:
    def __init__(self):
        print ("Engine started")
        self.name = "corolla"
        self.__make = "toyota"
        self._model = 1999
```

```
1  # --- Developer: Amine 2020-06-14 10:07
2  # --- Updated : Amine 2020-06-17 10:22
3  import json
4
5
6  class Currency(object):
7      __AUTO_id: int = 0
8
9      @classmethod
10     def __pick_next_id(cls) -> int:
11         Currency.__AUTO_id += 1
12         return Currency.__AUTO_id
13
14     def __init__(self, name: str, country: str):
15         self.id = Currency.__pick_next_id()
16         self.name = name
17         self.country = country
```

Méthode de l'instance

- Mot-clé **self** est présent parmi les paramètres.
- Symbol -> pour retourner un résultat.
- Les paramètres peuvent être typés (typage statique) ou non typés (typage dynamique)
- Accès aux variables d'instance avec le mot-clé self.
- La variable qui n'est pas précédée par self est locale.
- Une fonction peut retourner plus d'un résultat (un tuple)
- Les paramètres peuvent avoir une valeur par défaut.
- On peut affecter dynamiquement une méthode à une classe.

```
class Doer(object):
    def __init__(self):
        self.name = "Bob"

    def doSomething(self):
        print "%s got it done" % self.name

    def doItBetter(self):
        print "%s got it done better" % self.name

Doer.doSomething = doItBetter
```

```
1 # --- Developer: Amine 2020-06-15 22:01
2 from calforex.dal.idao import IRateDAO, ICurrencyDAO
3
4
5 class ExchangeService:
6     def __init__(self, dao: IRateDAO = None):
7         self.dao = dao
8
9     def convert(self, source: str, target: str, value: float) -> float:
10         print(f"Running conversion of the amount {value} from {source} to {target}")
11         rate = self.dao.fetch_rate(source, target)
12         print("The current Conversion rate is: " + str(rate))
13         return rate * value
14
15
16 class CurrencyService:
17     def __init__(self, dao: ICurrencyDAO = None):
18         self.dao = dao
19
20     def find_all_currencies(self):
21         print("Retrieving available currencies...")
22         self.dao.fetch_all_currencies()
23         pass
```

Méthode de classe (statique)

- Deux façons pour associer une méthode statique à une classe :
 - L'annotation `@classmethod` : la méthode de classe (statique) doit être annotée avec le décorateur **`@classmethod`** et avoir comme premier paramètre **`cls`**.
 - De façon dynamique en affectant une fonction à une classe avec l'instruction **`classmethod(theMethod)`**

```
class Shape:
    def info(msg):
        # show custom message
        print(msg)
        print("This class is used for representing different shapes.")
# create info static method
Shape.info = staticmethod(Shape.info)
Shape.info("Welcome to Shape class")
```

<https://www.studytonight.com/python/python-static-keyword>

```
1  # --- Developer: Amine 2020-06-14 10:07
2  # --- Updated : Amine 2020-06-17 10:22
3  import json
4
5
6  class Currency(object):
7      __AUTO_id: int = 0
8
9      @classmethod
10     def pick_next_id(cls) -> int:
11         Currency.__AUTO_id += 1
12         return Currency.__AUTO_id
13
14     def __init__(self, name: str, country: str):
15         self.id = Currency.pick_next_id()
16         self.name = name
17         self.country = country
```

Dunder methods:

`__init__`

- Python ne supporte pas la surcharge (Overloading) des fonctions. C'est toujours la dernière définition de la fonction qui est retenue (Overriding).
- Toutefois, une fonction peut avoir un nombre variable de paramètres.
- Ne pas confondre `__int__` et `__new__`. La fonction `__new__` est appelée quand l'objet est créé alors que `__init__` est appelée pour initialiser cet objet. `cls` représente la classe qui besoin d'être instanciée.

```
# Python program to
# demonstrate __new__

# don't forget the object specified as base
class A(object):
    def __new__(cls):
        print("Creating instance")
        return super(A, cls).__new__(cls)

    def __init__(self):
        print("Init is called")

A()
```

Creating instance
Init is called

<https://www.geeksforgeeks.org/new-in-python/>

```
1  # --- Developer: Amine 2020-06-15 22:01
2  from calforex.dal.idao import IRateDAO, ICurrencyDAO
3
4
5  class ExchangeService:
6      def __init__(self, dao: IRateDAO = None):
7          self.dao = dao
```

Dunder methods:

`__repr__` et `__str__`

- `__repr__` redéfinit la chaîne de caractères obtenue lorsqu'on sérialise l'objet.
- `__str__` donne aussi une représentation en chaîne de caractères d'un objet.

```
class User:
    def __init__(self, name, occupation):
        self.name = name
        self.occupation = occupation

    def __repr__(self):
        return f"{self.name} is a {self.occupation}"

u = User('John Doe', 'gardener')

print(f'{u}')
```

```
import datetime
today = datetime.datetime.now()

# Prints readable format for date-time object
print str(today)

# prints the official format of date-time object
print repr(today)

2016-02-22 19:32:04.078030
datetime.datetime(2016, 2, 22, 19, 32, 4, 78030)
```

Dunder methods:

`__del__` et `__delete__`

- `__del__` est le destructeur. Cette méthode est appelée avec l'instruction `del` ou automatiquement par le GC quand le nombre de références à l'objet est nul.
- `__delete__` permet d'effacer une variable d'une classe

<code>__delete__</code>	<code>__del__</code>
Used to delete attribute of owner class	Used as a destructor
Syntax: <code>def __delete__(self, instance):</code> NOTE: Parameters are compulsory	Syntax: <code>def __del__(self):</code> NOTE: Parameter is compulsory.
Need to be called explicitly	Called implicitly (explicitly also possible)
<code>del</code> keyword is used for calling with attribute as parameter	<code>del</code> may or may not be used as a parameter
Executed before program ends	Executed after the program ends

```
class Example(object):

    # Initializing
    def __init__(self):
        print("Example Instance.")

    # Calling __delete__
    def __delete__(self, instance):
        print ("Deleted in Example object.")

# Creating object of Example
# class as an descriptor attribute
# of this class
class Foo(object):
    exp = Example()

# Driver's code
f = Foo()
del f.exp

Example Instance.
Deleted in Example object.
```

Dunder methods: Comparaison des objets

- `__cmp__` n'est plus présente dans Python 3
- Avec le décorateur de classe **`functools.total_ordering`**, il suffit de définir deux Dunder méthodes: `__eq__` (resp. `__ne__`) et l'une des autres Dunder de comparaison (`__lt__`, `__le__`, `__gt__`, `__ge__`).

```
83 rates = [Rate(euro, cad, 1.53), Rate(euro, usd, 1.53),
84           Rate(cad, euro, 0.65)]
85 if rates[0] == rates[1]:
86     print("same rate")
87 else:
88     print("rate is diff.")
```

```
29 @total_ordering
30 class Rate(object):
31     __AUTO_id: int = 0
32
33     @classmethod
34     def __pick_next_id(cls) -> int:
35         Rate.__AUTO_id += 1
36         return Rate.__AUTO_id
37
38     def __init__(self, source: Currency,
39                 target: Currency, value: float):
40         self.id = Rate.__pick_next_id()
41         self.source = source
42         self.target = target
43         self.value = value
44
45     def __repr__(self):
46         return "{" + str(self.source) + "," + \
47             + str(self.target) + "," + str(self.value) + "}"
```

```
49     def __lt__(self, other):
50         # r1 < r2 calls r1.__lt__(r2)
51         return self.value < other.value
52
53     def __eq__(self, other):
54         # r1 == r2 calls r1.__eq__(r2)
55         return self.value == other.value
56
57     @property
58     def get_rate_value(self):
59         return self.value
```

Méthodes particulières de la classe:

`__dict__` et `vars`

- Tout objet a un attribut `__dict__` qui contient tous les attributs de l'objet avec les valeurs associées sous la forme d'un dictionnaire.
- La fonction `vars()` retourne ce dictionnaire.

```
79     euro = Currency("EUR0", "Europe")
80     cad = Currency("CAD", "Canada")
81     usd = Currency("USD", "USA")
82     rates = [Rate(euro, cad, 1.53), Rate(euro, usd, 1.53),
83             Rate(cad, euro, 0.65)]
84     print("Via __dict__ rates[0]=" + str(rates[0].__dict__))
85     print("Via vars      rates[0]=" + str(vars(rates[0])))
```

```
Via __dict__ rates[0]={'id': 1, 'source': {1,EUR0,Europe}, 'target': {2,CAD,Canada}, 'value': 1.53}
Via vars      rates[0]={'id': 1, 'source': {1,EUR0,Europe}, 'target': {2,CAD,Canada}, 'value': 1.53}
```

Héritage

- Python offre héritage multiple.
- Toute les classe héritent par défaut de la classe **object**. Indiquer la classe object est facultatif.
- Il n'y a pas de différence dans la déclaration entre dériver une classe et implémenter une interface.

```
1  # --- Developer: Amine 2020-06-13 23:14
2  import ...
7
8
9  class MongoAtlasDAO(NoSQLRepositoryDao, IRateDAO, ICurrencyDAO):
10
11     def __init__(self):
12         host = "colvalcluster-shard-00-00-cdr57.mongoddb.net:27017," \
13              "colvalcluster-shard-00-01-cdr57.mongoddb.net:27017," \
14              "colvalcluster-shard-00-02-cdr57.mongoddb.net:27017/test?"
15         user = "adbuser"
16         password = "adbuser2020"
17         db_name = "BDAW20"
18
19         # super(MongoAtlasDAO, self).__init__(host, user, password, db_name)
20         NoSQLRepositoryDao.__init__(self, host, user, password, db_name)
21         self.options = "ssl=true&replicaSet=ColvalCluster-shard-0&authSource=admin&retryWrites=true"
22         self.connection_string = "mongodb://" + self.user + ":" + self.password \
23                                 + "@" + self.host + self.options
24         self.client = None
25         self.connect()
26         self.db = self.client[db_name]
27         self.calforex_collection = self.db["Calforex"]
```

docstrings

- La chaîne de caractères entre triple guillemets située au début d'un module, une classe ou d'une fonction.
- Docstring est stockée dans un attribut `__doc__` que l'on peut utiliser pour récupérer cette chaîne.
- Les docstrings permettent notamment de fournir de l'aide lorsqu'on invoque la commande `help()`.

```
def add_binary(a, b):  
    '''  
    Returns the sum of two decimal numbers in binary digits.  
  
    Parameters:  
        a (int): A decimal integer  
        b (int): Another decimal integer  
  
    Returns:  
        binary_sum (str): Binary string of the sum of a and b  
    '''  
    binary_sum = bin(a+b)[2:]  
    return binary_sum  
  
print(add_binary.__doc__)
```

Output

```
Returns the sum of two decimal numbers in binary digits.  
  
Parameters:  
    a (int): A decimal integer  
    b (int): Another decimal integer  
  
Returns:  
    binary_sum (str): Binary string of the sum of a and b
```

Styles de docstrings



```
"""
This is an example of Google style.

Args:
    param1: This is the first param.
    param2: This is a second param.

Returns:
    This is a description of what is returned.

Raises:
    KeyError: Raises an exception.
"""
```



```
"""
This is a reST style.

:param param1: this is a first param
:param param2: this is a second param
:returns: this is a description of what is returned
:raises keyError: raises an exception
"""
```



```
Parameters
-----
first : array_like
    the 1st param name 'first'
second :
    the 2nd param
third : {'value', 'other'}, optional
    the 3rd param, by default 'value'

Returns
-----
string
    a value in a string

Raises
-----
KeyError
    when a key error
OtherError
    when an other error
"""
```

Les collections Python : la liste

- Python autorise la construction de liste contenant des valeurs de types différents.
- Les indices d'une liste de n éléments commence à 0 et se termine à $n-1$.
- La liste peut également être indexée avec des nombres négatifs
- On peut sélectionner une partie d'une liste en utilisant un indicage construit sur le modèle $[m:n+1]$ pour récupérer tous les éléments, du m ème au n ème (de l'élément m inclus à l'élément $n+1$ exclu).
- On peut aussi préciser le pas en ajoutant un symbole deux-points supplémentaire et en indiquant le pas par un entier.
- Plusieurs méthodes applicables: len, min, max, sum, append, etc.

```
1 liste : ['A', 'B', 'C', 'D', 'E', 'F']
2 indice positif : 0 1 2 3 4 5
3 indice négatif : -6 -5 -4 -3 -2 -1
```

```
1 >>> animaux = ['girafe', 'tigre', 'singe', 'souris']
2 >>> animaux[0:2]
3 ['girafe', 'tigre']
```

```
1 >>> animaux = ['girafe', 'tigre', 'singe', 'souris']
2 >>> animaux[0:3:2]
3 ['girafe', 'singe']
```

Python List Methods	
Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Les collections Python :

Trier une liste d'objets

- Le tri est réalisé par la méthode membre sort ou la fonction sorted.
- La méthode sort prend deux paramètres : l'ordre et une fonction de tri.
- L'ordre par défaut est ascendant
- La fonction de tri peut être écrite comme une fonction Python ordinaire ou sous la forme lambda.

```
46 # --- Developer:      Amine 2020-06-17 12:55
47 # --- Last update :  Amine 2020-06-18 20:27
48
49 class CarComparator:
50
51     @classmethod
52     def compare_year(cls, car):
53         return car['year']
54
55     @classmethod
56     def compare_make_length(cls, car):
57         return len(car["make"])
58
59 def test_comparator():
60     cars = [
61         {'make': 'Ford', 'year': 2005},
62         {'make': 'Mitsubishi', 'year': 2000},
63         {'make': 'BMW', 'year': 2019},
64         {'make': 'VW', 'year': 2011}
65     ]
66     cars.sort(key=CarComparator.compare_year)
67     print(cars)
68     cars.sort(reverse=True, key=CarComparator.compare_year)
69     print(cars)
70     cars.sort(key=CarComparator.compare_make_length)
71     print(cars)
72     cars.sort(key=lambda c: c['year'])
73     print(cars)
```

```
[{'make': 'Mitsubishi', 'year': 2000}, {'make': 'Ford', 'year': 2005}, {'make': 'VW', 'year': 2011}, {'make': 'BMW', 'year': 2019}]
[{'make': 'BMW', 'year': 2019}, {'make': 'VW', 'year': 2011}, {'make': 'Ford', 'year': 2005}, {'make': 'Mitsubishi', 'year': 2000}]
[{'make': 'VW', 'year': 2011}, {'make': 'BMW', 'year': 2019}, {'make': 'Ford', 'year': 2005}, {'make': 'Mitsubishi', 'year': 2000}]
[{'make': 'Mitsubishi', 'year': 2000}, {'make': 'Ford', 'year': 2005}, {'make': 'VW', 'year': 2011}, {'make': 'BMW', 'year': 2019}]
```

Trier une liste d'objets (sorted)

```
7 class Currency(object):
8     __AUTO_id: int = 0
9
10    @classmethod
11    def __pick_next_id(cls) -> int:
12        Currency.__AUTO_id += 1
13        return Currency.__AUTO_id
14
15    def __init__(self, name: str, country: str):
16        self.id = Currency.__pick_next_id()
17        self.name = name
18        self.country = country
```

```
29 @total_ordering
30 class Rate(object):
31     __AUTO_id: int = 0
32
33    @classmethod
34    def __pick_next_id(cls) -> int:
35        Rate.__AUTO_id += 1
36        return Rate.__AUTO_id
37
38    def __init__(self, source: Currency,
39                 target: Currency, value: float):
40        self.id = Rate.__pick_next_id()
41        self.source = source
42        self.target = target
43        self.value = value
```

```
60 # --- Developer: Amine 2020-06-14 10:07
61 # --- Updated : Amine 2020-06-18 10:24
62
63 class RateComparator:
64     @classmethod
65     def compare_rate_value(cls, rate: Rate):
66         return rate.value
67
68     @classmethod
69     def compare_currency(cls, rate: Rate):
70         return rate.source.name
71
72     @classmethod
73     def compare_country(cls, rate: Rate):
74         return rate.source.country
```

```
77 euro = Currency("EUR", "Europe")
78 cad = Currency("CAD", "Canada")
79 usd = Currency("USD", "USA")
80
81 rates = [Rate(euro, cad, 1.53), Rate(euro, usd, 1.13),
82          Rate(cad, euro, 0.65)]
83
84 print("Calling sorted using lambda: " +
85       str(sorted(rates, key=lambda rate: rate.value)))
86 print(sorted(rates, key=RateComparator.compare_rate_value))
87 print(sorted(rates, key=RateComparator.compare_currency))
88 print(sorted(rates, key=RateComparator.compare_country))
```

```
Calling sorted using lambda: [{2,CAD,Canada},{1,EUR,Europe},0.65], [{1,EUR,Europe},{3,USD,USA},1.13], [{1,EUR,Europe},{2,CAD,Canada},1.53]
[{{2,CAD,Canada},{1,EUR,Europe},0.65}, {{1,EUR,Europe},{3,USD,USA},1.13}, {{1,EUR,Europe},{2,CAD,Canada},1.53}]
[{{2,CAD,Canada},{1,EUR,Europe},0.65}, {{1,EUR,Europe},{2,CAD,Canada},1.53}, {{1,EUR,Europe},{3,USD,USA},1.13}]
[{{2,CAD,Canada},{1,EUR,Europe},0.65}, {{1,EUR,Europe},{2,CAD,Canada},1.53}, {{1,EUR,Europe},{3,USD,USA},1.13}]
```

Les collections Python : l'ensemble

- Une collection de données qui ne sont ni indexés ni ordonnés et sont écrits avec des accolades ({}).

```
car_models = {"Mustang", "Camaro", "Civic", "Jetta"}
for x in car_models:
    print(f"Car model: {x} ")
```

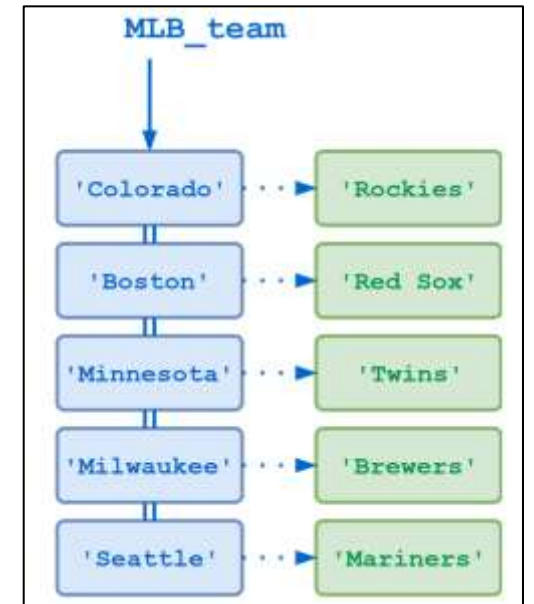
- Plusieurs méthodes telles que: len, add, update, remove, discard, etc.

Python Sets Methods	
Method	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
copy()	Returns a copy of the set
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()	Returns whether two sets have a intersection or not
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
remove()	Removes the specified element
symmetric_difference()	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	inserts the symmetric differences from this set and another
union()	Return a set containing the union of sets
update()	Update the set with the union of this set and others

Les collections Python : le dictionnaire

- Si on a besoin d'une structure de données séquentielles ou si on veut donner des noms aux variables (plutôt qu'un indice), utilisez plutôt les dictionnaires.
- Une collection de clé-valeur
- Trois façons pour créer un dictionnaire comme illustré ci-bas.

```
114 # --- Developer: Amine 2020-06-14 18:07
115 # --- Updated : Amine 2020-06-19 00:03
116
117 mlb_teams_west = dict([('Colorado', 'Rockies'), ('Los Angeles', 'Angels'), ('Seattle', 'Mariners')])
118 mlb_teams_east = {'New York': 'Yankees', 'Boston': 'Red Sox', 'Toronto': 'Blue Jays'}
119 mlb_teams_central = dict(Detroit='Tigers', Minnesota='Twins', Cleveland='Indians')
120 teams = {**mlb_teams_west, **mlb_teams_east}
121 teams.update(mlb_teams_central)
122 teams['Chicago'] = 'White Sox'
123 print(teams)
124 print(teams.keys())
125 print(teams.values())
```



```
{'Colorado': 'Rockies', 'Los Angeles': 'Angels', 'Seattle': 'Mariners', 'New York': 'Yankees',  
'Boston': 'Red Sox', 'Toronto': 'Blue Jays', 'Detroit': 'Tigers', 'Minnesota': 'Twins',  
'Cleveland': 'Indians', 'Chicago': 'White Sox'}
```

```
dict_keys(['Colorado', 'Los Angeles', 'Seattle', 'New York', 'Boston', 'Toronto', 'Detroit', 'Minnesota', 'Cleveland', 'Chicago'])  
dict_values(['Rockies', 'Angels', 'Mariners', 'Yankees', 'Red Sox', 'Blue Jays', 'Tigers', 'Twins', 'Indians', 'White Sox'])
```

Les collections Python :

le tuple

- Les tuples (« n-uplets » en français) correspondent aux listes à la différence qu'ils sont non modifiables. Ils utilisent les parenthèses au lieu des crochets.
- On peut construire un tuple à partir d'une liste avec **tuple([e1, e2, ...])**
- On peut construire un tuple directement avec (e1, e2, ...)

```
79 euro = Currency("EURO", "Europe")
80 cad = Currency("CAD", "Canada")
81 usd = Currency("USD", "USA")
82 currencies = tuple([euro, cad, usd])
83 print(currencies[0:3])
84 rates = (Rate(euro, cad, 1.53), Rate(euro, usd, 1.53))
85 rates = rates + (Rate(cad, euro, 0.65), )
86 for i, r in enumerate(rates):
87     print(i, r)
```

```
({1,EURO,Europe}, {2,CAD,Canada}, {3,USD,USA})
0  {{1,EURO,Europe},{2,CAD,Canada},1.53}
1  {{1,EURO,Europe},{3,USD,USA},1.53}
2  {{2,CAD,Canada},{1,EURO,Europe},0.65}
```

Les itérateurs :

- Un itérateur est un objet qui contient un nombre dénombrable de valeurs. Il peut être itéré, ce qui signifie que vous pouvez parcourir toutes les valeurs.
- En Python, un itérateur est un objet qui implémente le protocole d'itérateur composé des trois méthodes `__init__()`, `__next__()` et `__iter__()`.
- On peut appliquer ce protocole à n'importe quel conteneur ayant des valeurs/objets.

```
34 class Currency(object):
35     currency_id_generator = IDGenerator(0, 100)
36
37     @classmethod
38     def __pick_next_id(cls) -> int:
39         return Currency.currency_id_generator.__next__()
40
41     def __init__(self, name: str, country: str):
42         self.id = Currency.__pick_next_id()
43         self.name = name
44         self.country = country
```

```
11 # --- Developer: Amine 2020-06-14 10:07
12 # --- Updated : Amine 2020-06-19 11:31
13
14
15 class IDGenerator(object):
16
17     def __init__(self, start: int = 0, limit: int = 100):
18         self.current_id = start
19         self.limit = limit
20
21     def __next__(self):
22         next_id: int = self.current_id + 1
23         if next_id == self.limit:
24             raise IDHasReachedItsLimitException\
25                 ("ID has reached the sky")
26         else:
27             self.current_id = next_id
28             return next_id
29
30     def __iter__(self):
31         return self
```

Les exceptions



- Si une exception se produit, mais que la clause except dans le code ne la gère pas, elle est transmise aux instructions try externes. Si l'exception n'est pas gérée, l'exécution s'arrête.
- L'instruction raise permet de forcer une exception spécifique à se produire.

```
# Function which returns a/b
def AbyB(a , b):
    try:
        c = ((a+b) / (a-b))
    except ZeroDivisionError:
        print "a/b result in 0"
    else:
        print c
```

```
# Program to depict Raising Exception

try:
    raise NameError("Hi there")  # Raise Error
except NameError:
    print "An exception"
    raise  # To determine whether the exception was raised or not
```

Notions Python supplémentaires:

Tests des valeurs sur des floats

- Les valeurs de **float** ne peuvent être qu'approchées. Une manière de s'en rendre compte est d'utiliser l'écriture formatée en demandant l'affichage d'un nombre variable de décimales.
- Il ne faut surtout pas tester si un **float** est égal à une certaine valeur.
- La bonne pratique est de vérifier si un float est compris dans un intervalle avec une certaine précision. Si on appelle cette précision delta, on peut procéder comme illustrer ci-contre

```
1 >>> 1/10 == 0.1
2 True

1 >>> (3 - 2.7) == 0.3
2 False
3 >>> 3 - 2.7
4 0.29999999999999998

1 >>> delta = 0.0001
2 >>> var = 3.0 - 2.7
3 >>> 0.3 - delta < var < 0.3 + delta
4 True
5 >>> abs(var - 0.3) < delta
6 True
```

Notions Python supplémentaires: manipuler une chaîne de caractères

- Considérée comme une liste **non modifiable** de caractères (Java String)
- Peut être manipuler avec l'opérateur []
 - [3] est le 3^{ème} caractère dans la chaîne.
 - [-3:] les trois derniers caractères
 - [0:4] les 4 premiers caractères
 - [9:] les caractères à partir de la 9^{ème} position jusqu'à la fin
 - [:-2] tous les caractères de la chaîne sauf les deux derniers caractères
 - [0:] = [:len(name)] la chaîne entière
- Les fonctions int(), float(), str() permettent de passer d'un nombre vers une chaîne de caractères et vice versa.

Notions Python supplémentaires: manipuler une chaîne de caractères

STRING METHOD	WHAT IT DOES
<code>s.center(width)</code>	Returns a copy of <code>s</code> centered within the given number of columns.
<code>s.count(sub [, start [, end]])</code>	Returns the number of non-overlapping occurrences of substring <code>sub</code> in <code>s</code> . Optional arguments <code>start</code> and <code>end</code> are interpreted as in slice notation.
<code>s.endswith(sub)</code>	Returns <code>True</code> if <code>s</code> ends with <code>sub</code> or <code>False</code> otherwise.
<code>s.find(sub [, start [, end]])</code>	Returns the lowest index in <code>s</code> where substring <code>sub</code> is found. Optional arguments <code>start</code> and <code>end</code> are interpreted as in slice notation.
<code>s.isalpha()</code>	Returns <code>True</code> if <code>s</code> contains only letters or <code>False</code> otherwise.
<code>s.isdigit()</code>	Returns <code>True</code> if <code>s</code> contains only digits or <code>False</code> otherwise.

STRING METHOD	WHAT IT DOES
<code>s.join(sequence)</code>	Returns a string that is the concatenation of the strings in the sequence. The separator between elements is <code>s</code> .
<code>s.lower()</code>	Returns a copy of <code>s</code> converted to lowercase.
<code>s.replace(old, new [, count])</code>	Returns a copy of <code>s</code> with all occurrences of substring <code>old</code> replaced by <code>new</code> . If the optional argument <code>count</code> is given, only the first <code>count</code> occurrences are replaced.
<code>s.split([sep])</code>	Returns a list of the words in <code>s</code> , using <code>sep</code> as the delimiter string. If <code>sep</code> is not specified, any whitespace string is a separator.
<code>s.startswith(sub)</code>	Returns <code>True</code> if <code>s</code> starts with <code>sub</code> or <code>False</code> otherwise.
<code>s.strip([aString])</code>	Returns a copy of <code>s</code> with leading and trailing whitespace (tabs, spaces, newlines) removed. If <code>aString</code> is given, remove characters in <code>aString</code> instead.
<code>s.upper()</code>	Returns a copy of <code>s</code> converted to uppercase.

Notions Python supplémentaires: f-strings (depuis 3.6)

- Permettent de formater une chaîne de caractères.
- f-string utilise le prefix f' et une expression entre {} à évaluer.

```
bags = 3
apples_in_bag = 12
print(f'There are total of {bags * apples_in_bag} apples')
```

```
user = {'name': 'John Doe', 'occupation': 'gardener'}
print(f"{user['name']} is a {user['occupation']}")
```

```
print(f'Max of {a} and {b} is {mymax(a, b)}')
```

```
for currency in currencies:
    print("{0} {1}".format(currency["Source.Name"], currency["Target.Name"]))
```

```
msg = (
    f'Name: {name}\n'
    f'Age: {age}\n'
    f'Occupation: {occupation}'
)
print(msg)
```

```
val = 12.3
print(f'{val:.2f}')
print(f'{val:.5f}')
```

Notions Python supplémentaires

Avoir plusieurs constructeurs

```
from datetime import date

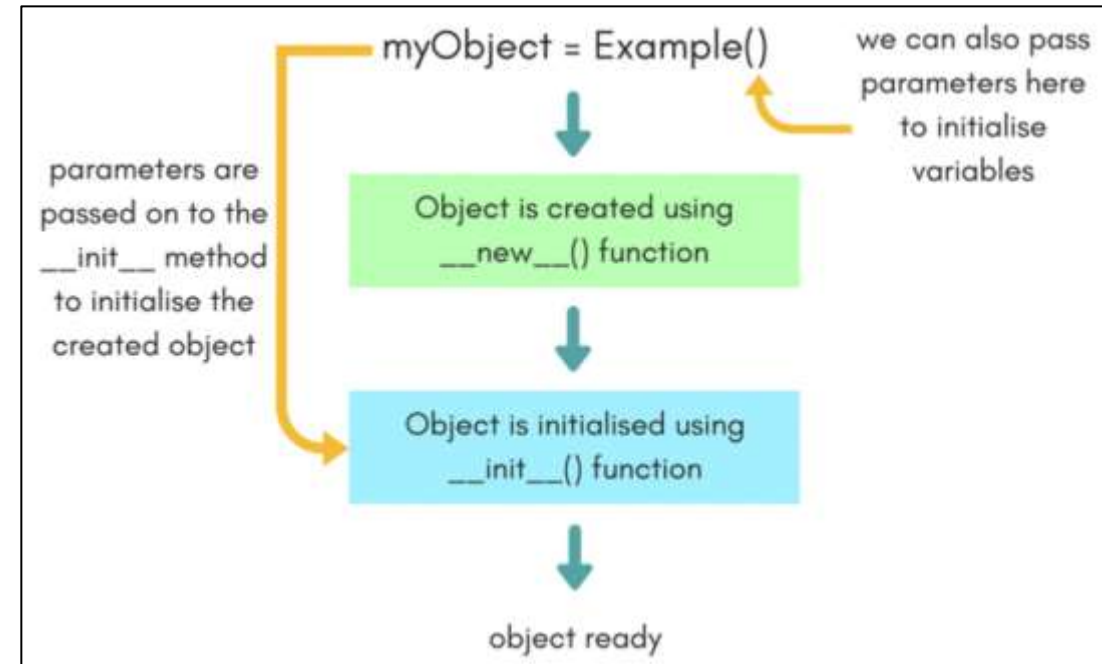
# random Person
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @staticmethod
    def fromFathersAge(name, fatherAge, fatherPersonAgeDiff):
        return Person(name, date.today().year - fatherAge + fatherPersonAgeDiff)

    @classmethod
    def fromBirthYear(cls, name, birthYear):
        return cls(name, date.today().year - birthYear)

    def display(self):
        print(self.name + "'s age is: " + str(self.age))
```

<https://www.programiz.com/python-programming/methods/built-in/classmethod>



<https://www.studytonight.com/python/constructors-in-python>

Conventions de nommage

```
1 # --- Developer: Amine 2020-06-13 23:14
2
3 import pymongo as pymongo
4
5 from Sources.bll.model import Rate
6 from Sources.dal.idao import IRateDAO, ICurrencyDAO
7 from Sources.dal.NoSQLRepositoryDAO import NoSQLRepositoryDao
8
9
10 class MongoAtlasDAO(NoSQLRepositoryDao, IRateDAO, ICurrencyDAO):
```

```
48 def fetch_rate(self, source: str, target: str) -> float:
49     """Overrides ICurrencyDAO.fetch_rate()"""
50     print("Fetching required rate in progress...")
51     query = {"$and": [{"Source.Name": source}, {"Target.Name": target}]}
52     found = self.calforexCollection.find(query)[0]
53     print(found)
54     return found["Rate"]
55
56 def fetch_all_rates(self) -> []:
57     """Overrides ICurrencyDAO.fetch_all_rates()"""
58     print("Fetching all rates in progress...")
59     rates = []
60     for doc in self.calforexCollection.find():
61         # print("Objectifying the following json document: ")
62         # print(doc)
63         rates.append(Rate(doc["Source"]["Name"], doc["Target"]["Name"], doc["Rate"]))
64     return rates
```

Type	Public	Internal
Packages	lower_with_under	
Modules	lower_with_under	_lower_with_under
Classes	CapWords	_CapWords
Exceptions	CapWords	
Functions	lower_with_under()	_lower_with_under()
Global/Class Constants	CAPS_WITH_UNDER	_CAPS_WITH_UNDER
Global/Class Variables	lower_with_under	_lower_with_under
Instance Variables	lower_with_under	_lower_with_under
Method Names	lower_with_under()	_lower_with_under()
Function/Method Parameters	lower_with_under	
Local Variables	lower_with_under	

Top 10 Python modules

- NumPy (calcul numérique)
- Pendulum (manipulation des dates et du temps)
- Python Imaging Library
- MoviePy (video editing)
- Requests (HTTP requests)
- Tkinter (Tk GUI toolkit)
- PyQt (cross-platform GUI toolkit Qt)
- Pandas (data analysis and manipulation tool)
- Pywin32 (Windows API functions)
- Pytest (Unit testing)



Références

- <https://realpython.com/>
- <https://www.programiz.com/python-programming>
- <http://zetcode.com/lang/python/>